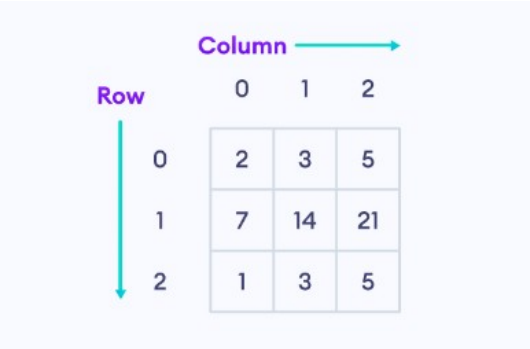
CSCI160 Computer Science I
Lab Exercise: Matrix Processing

In mathematics, a matrix (plural matrices) is a rectangular array (list) or table of numbers, symbols, or expressions, arranged in rows and columns.



For example, the dimension of the matrix below is 2 × 3 (read "two by three"), because there are two rows and three columns:

```
[ 32, 98, -1 ]
[ 17,  4, 21 ]
```

The concept of a matrix is implemented in programming languages with a ***nested list***. The matrix above would be implemented in Python with the following list definition, where each element in the list ***matrix*** is also a list.

```
matrix = [
     [ 32, 98, -1 ], # row 0
     [ 17,  4, 21 ], # row 1
]
```

With the matrix defined you can access a matrix element using two index values: ***one for the row, one for the column***

```
matrix[0][0] => 32  # row 0, column 0
matrix[0][1] => 98  # row 0, column 1
matrix[1][0] => 17  # row 1, column 0
matrix[1][2] => 21  # row 1, column 2
```

You can iterate through a matrix element by element using a set of nested for loops

```
for i in range( len( matrix ) ):          # number of rows
    for j in range( len( matrix[ i ] )): # num columns on row[i]
          matrix[ i ][ j ]               # process matrix[i][j]
```

Your task in this lab is to implement some basic matrix operations

- **Adding two matrices:** performing an element by element addition into a new matrix
- **Transposing a matrix:** reversing the rows and columns
- **Computing a column sum:** Summing a single column
- **Computing a row sum:** Summing a single row
- **Testing for conformability:** Do two matrices share the same number of rows and columns?
- **Testing for validity:** Are a matrix's rows all the same length?
- **Equality:** Are two matrices equal? Do they contain the same elements in the same positions?

Define the following functions. Type hints, comprehensive docstrings and 3 doctests are required for each definition. The functions should only perform their defined tasks. No input/output statements are allowed ***unless that is the functions job.*** All functions need to adhere to perform according to the signatures included in the lab. This will be strictly enforced in the grading.

When you submit your work you are required to submit screen shots clearly showing the doctests passing.

---

is_valid(matrix: list[list[int]]) -> bool

This function will accept a matrix and perform a validity test. A matrix is valid if all of it's rows are the same length. This function should return a Boolean

Example:
The matrix below is valid. All rows are the same length

```
[ 1,  2,  3,  4 ]
[ 5,  6,  7,  8 ]
[ 9, 10, 11, 12 ]
```

Example:
This matrix is not valid. Rows are not the same length

```
[ 1,  2,  3,  4]
[ 5,  6,  7,  8, 11, 23, 45]
[ 9, 10, 11]
```

---

are_conformable(m1: list[list[int]], m2: list[list[int]]) -> bool

---

This function will accept two matrices and determine if they are conformable. Two matrices are conformable if they have the same number of rows and columns. This function should return a Boolean

Example:
These two matrices are conformable, row and column lengths match
```
[1, 2, 3, 4]       [12, 34, 56, 78]
[5, 6, 7, 8]       [11, 12, 23, 34]
```

Example:
These two matrices are not, one has more columns
```
[1, 2, 3, 4]       [10, 12, 34]
[5, 6, 7, 8]       [54, 32, 21]
```

---

add(m1: list[list[int]], m2: list[list[int]]) -> list[list[int]]

This function will accept two matrices and perform element by element addition into a new matrix. This function should return a new matrix containing the addition.

Example:

```
[ 1, 2, 3 ]   +  [ 9, 5, 1 ]  =  [ 10,  7,  4 ]
[ 4, 5, 6 ]      [ 1, 9, 4 ]     [  5, 14, 10 ]
```

---

equal(m1: list[list[int]], m2: list[list[int]]) -> bool

This function will accept two matrices and perform an element by element comparison. If any of the elements are not equal the function will return False. If all elements are equal the function will return True. This function returns a Boolean

Example: The matrices below are equal
```
[ 1, 2, 3 ]     [ 1, 2, 3 ]
[ 4, 5, 6 ]     [ 4, 5, 6 ]
```

Example: The matrices below are not equal
```
[ 1, 2, 3 ]     [ 1, 2, 3 ]
[ 4, 5, 6 ]     [ 4, 1, 6 ]
```

---

transpose(matrix: list[list[int]]) -> list[list[int]]

This function will accept a matrix and transpose it into a new matrix. Transposing a matrix takes the rows and makes them columns

Example: The 3 x 4 matrix below is transposed to a 4 x 3 matrix

```
[ 1,  2,  3,  4]  -> is transposed to ->   [ 1, 5, 9  ]
[ 5,  6,  7,  8]                           [ 2, 6, 10 ]
[ 9, 10, 11, 12]                           [ 3, 7, 11 ]
                                           [ 4, 8, 12 ]
```

---

column_sum(matrix: list[list[int]], col_num:int) -> int

This function will accept a matrix and an integer representing the column number (1 based). The function will then compute the sum of the numbers in that column. If the column number is out of range, the function should return zero.

Example: The sum of column 1 for this matrix is 15 (1 + 5 + 9)

```
[ 1,  2,  3,  4 ]
[ 5,  6,  7,  8 ]
[ 9, 10, 11, 12 ]
```

---

row_sum(matrix: list[list[int]], row_num: int) -> int

This function will accept a matrix and an integer representing the row number (1 based). The function will then compute the sum of the numbers in that row. If the row number is out of range, the function should return zero.

Example: The sum of row 2 for this matrix is 26 (5 + 6 + 7 + 8)

```
[ 1,  2,  3,  4 ]
[ 5,  6,  7,  8 ]
[ 9, 10, 11, 12 ]
```

---

print_matrix(matrix: list[list[int]]) -> None

```
This function will accept a single nested list and print it in a nice
format. Each element should be printed in a right justified column of
width 2. There is a space after each '[' and a space before each ']'

Example: The following list

matrix = [
    [1, 2, 3, 4, 5 ],
    [6, 7, 8, 9, 10],
    [11,12,13,14,15]
]

would be printed as

[  1  2  3  4  5 ]
[  6  7  8  9 10 ]
[ 11 12 13 14 15 ]
```

You have been provided with **main.py** that contains all the input and output operations. Download it and play with it to make sure you understand what it is doing. Use this to test your code but you are not allowed to modify it and do not submit it. The original copy will be used to test your code and it must work perfectly with the provided main.

The only output formatting you have to adhere to is the **print_matrix** function

Rubric is on the next page

RUBRIC

Category	Criteria	Points
Functionality	All 7 required functions are implemented correctly and meet the specification - Each function returns correct results on all valid and edge cases (2 pts each for 7 functions)	14
Doctests	Each function includes at least 3 valid and meaningful doctests - Doctests are consistent with expected function behavior	3
Docstrings	Each function has a clear, complete docstring - Includes purpose, parameters, return type, and example if appropriate	3
Type Hints	Each function has correct Python type hints for parameters and return value	2
Design and Style	Code follows clear and consistent formatting - Uses appropriate naming, indentation, whitespace, and comments where needed	2
Function Scope & Separation	No I/O statements are present (except in print_matrix) - Each function is clearly focused on a single task	2
Testing Evidence	Screenshot(s) submitted showing all doctests passing	2
print_matrix Formatting	print_matrix matches the format exactly as described in the spec	2